

BTC-Predictor

Trading Bot Architecture & Design Report

Generated from BTC_Predictor_Bot_Design_Report.md · Repository: btc-predictor

BTC-Predictor — Comprehensive Design & Operations Report

Version: June 2026 · main branch

Audience: Operator review — architecture, ML, bots, dashboard, risks, tuning

Companion PDF: docs/BTC_Predictor_Bot_Design_Report.pdf (regenerate via python3 scripts/generate_design_report_pdf.py)

Part I — System Overview

1.1 What This Application Does

BTC-Predictor is an end-to-end system for:

1. **Predicting** short-horizon Bitcoin/Ethereum direction using LightGBM models on OHLCV features
2. **Mapping predictions** to Kalshi binary contracts (15-minute slots, hourly threshold brackets, range bands)
3. **Auto-trading** six independent bots with paper or live execution
4. **Reporting** P&L, calibration, and operational health on a web dashboard

The trading layer is **rule-heavy by design**: ML produces probabilities and signals; bots apply edge gates, Kelly sizing, regime filters, cooldowns, and risk caps before any order.

1.2 Six Independent Bots

Bot	Period	Config	DB file
BTC Hourly	1 hour	<code>`hourly.bot`</code>	<code>`hourly_bot_btc.db`</code>
BTC Hourly Trial	1 hour	<code>`hourly.bot` + `trial`</code>	<code>`hourly_trial_bot_btc.db`</code>
BTC 15m	15 min	<code>`intra_slot.bot`</code>	<code>`slot15_bot_btc.db`</code>
ETH Hourly	1 hour	<code>`eth.hourly.bot`</code>	<code>`hourly_bot_eth.db`</code>
ETH Hourly Trial	1 hour	<code>`eth.hourly.bot.trial`</code>	<code>`hourly_trial_bot_eth.db`</code>
ETH 15m	15 min	<code>`eth.intra_slot.bot`</code>	<code>`slot15_bot_eth.db`</code>

Hourly trial uses the same HourlyBot class but slot15-style **leg stop / leg take-profit** exits (bird-in-hand) instead of thesis-based hourly alerts.

1.3 Continuous Bot Cycle (Every ``poll_seconds``, default 10s)

1. Detect hour/slot rollover → force-close prior period legs at settlement
2. Sync daily loss cap → may set `auto_stopped`
3. Live hygiene → adopt Kalshi orphans, cancel stale resting orders
4. PROCESS EXITS (all open legs)
5. Auto-stop if budget exhausted
6. PROCESS ENTRIES (ranked candidates, up to `max_entries_per_cycle`)

Design choice: Exits before entries prevents stacking risk on positions that should already be closed.

Part II — Machine Learning & Prediction

2.1 Model Inventory

Model	Trainer	Artifact	Inference
15m slot	`src/models/trainer.py`	`data/models/model.joblib`	`src/models/predictor.py`
Hourly	`src/models/hourly_trainer.py`	`data/models/model_hourly.joblib`	`src/models/hourly_predictor.py`
2nd Chance	`src/models/second_chance_trainer.py`	`data/models/model_second_chance.joblib`	`src/trading/second_chance.py`

Default algorithm: **LightGBM** (300 trees, lr=0.05, max_depth=6). Baseline heuristic used if no model file exists.

2.2 Training Data & Labels

15-minute slot model

- **Features:** 15m OHLCV (min 48 bars / 12h context), optional 1m micro-features, auxiliary (funding, NQ), session flags, waveform/structure features
- **Label:** next 15m slot close > current slot close (ET boundaries :00/:15/:30/:45)
- **Split:** chronological 80/20 (no shuffle — avoids leakage)
- **Min samples:** 1,500 rows after dropna

Hourly model

- **Features:** 1h candles (~720 bars / ~30 days), optional prob_15m_aggregate (mean of last 4 resolved 15m probs)
- **Label:** next hour close > current hour close
- **Min samples:** 500

2nd Chance model

- **Features (12):** open_prob_up, path stats at t+4min (gap, time above ref, crossings, momentum), elapsed/remaining minutes
- **Label:** at minute 4, will slot settle above t=0 reference?
- **Min samples:** 300

2.3 Retrain Schedule

Job	When	What
Full retrain	Daily **2:00 AM ET** (`auto_train`)	All enabled models (BTC + ETH) in parallel threads
Isotonic recalibration	Every **6 hours** + on new resolutions	Refit probability calibrator from production outcomes (trees unchanged)
Hourly sigma scale	On hourly resolve	Adjust terminal σ from realized vs forecast

Job	When	What
Bot auto-tune	Daily **3:00 AM ET**	Passive bots only: adjust min_ask_edge, kelly_fraction from trade log
Adaptive calibration	Every **30 min**	Bucket pause/probe from recent trade P&L

Trade-off: Full daily retrain adapts to recent market structure but can overfit thin regimes; isotonic refit is safer but only fixes probability calibration, not feature drift.

2.4 Feature Engineering Highlights

From `src/features/engineering.py`:

- Momentum windows on 15m: 1h, 2h, 4h, 8h, 12h lookbacks
- Volatility (48-bar), RSI(14), VWAP distance
- Volume spike vs 4h baseline
- Phase-2: velocity, acceleration, compression, liquidity sweeps
- **Open drive** (live only): first 1m bars at slot open

Columns kept if $\geq 85\%$ non-null coverage in training.

2.5 Probability Calibration (ML layer)

Isotonic regression on held-out test split at train time; refit online from resolved predictions:

- Needs ≥ 30 resolved samples with both classes
- Output clamped to $[0.02, 0.98]$
- 15m: prob_up vs Kalshi outcome (optional `calibration.kalshi_only` filter)
- Hourly: primary contract model_prob vs binary outcome

This is separate from `bot_adaptive_calibration` (trade P&L buckets) — see §2.7.

2.6 Prediction Pipelines & Blend Weights

15m slot (every :00/:15/:30/:45 ET)

LightGBM $\rightarrow$ isotonic $\rightarrow$ EdgeCalculator $\rightarrow$ LONG/SHORT/NO TRADE
 $\rightarrow$ RegimeFilter may veto (rule-based, not ML)

- min_edge_confidence: 0.57 default
- no_trade_band: 0.03

Hourly (:05 ET official lock)

ML branch: 1h features $\rightarrow$ LightGBM $\rightarrow$ isotonic $\rightarrow$ ml_mu
 Structure: DailyPredictor vol/drift + Kalshi book $\rightarrow$ structure_mu
 Blend: $\text{blended_mu} = 0.6 \times \text{ml_mu} + 0.4 \times \text{structure_mu}$ (configurable)

- **:00 ET** — hour-open preview (not scored for calibration)
- **:45 ET** — late-call snapshot (trading guidance only)

2nd Chance (:04/:19/:34/:49 ET)

$\text{blended_prob} = 0.55 \times \text{ML@t+4} + 0.25 \times \text{open_prob} + 0.20 \times \text{path_heuristic}$

Signals: 2ND LONG / 2ND SHORT / 2ND NO TRADE at min_confidence 0.57.

2.7 Adaptive Calibration — “Memory” of Losing Buckets

File: bot_adaptive_calibration.py

Purpose: Short-term **trade memory** — pause or tighten entries in price buckets that recently lost money.

Price buckets: 1–20¢, 21–40¢, 41–60¢, 61–80¢, 81–99¢

Window	Trigger pause	Pause duration
Short (4h)	≥4 trades, WR≤25%, loss≥\$2	6 hours
Long (24h)	≥8 trades, WR≤35%, loss≥\$5	24 hours

States: normal → tightened (+4¢ edge boost) → paused → probing (2 trial entries) → back to normal or re-pause

Trade-offs:

- **Upside:** Stops repeating known-bad entry zones (e.g. cheap lottery legs)
- **Downside:** Can over-react to small samples; reduces frequency in volatile hours
- **Aggressive mode:** bucket logic may still apply if `apply_in_aggressive_mode: true` (default)
- **15m bots:** adaptive calibration **off by default** (hour-scale memory hurts 15m frequency)

2.8 Regime Filter (Prediction layer, not ML)

Rule-based veto before bot sees signal. Hourly needs ≥2 of:

- Low expected move (`< min_expected_move_pct`)
- Late settle (`< min_hours_to_settle`)
- High sigma (`> max_sigma_pct`)
- Low edge (`< min_edge`)
- Compression

In **FREE mode** on dashboard: regime is **advisory only** — bot can still enter if ask-edge passes.

Part III — Entry Logic, Math & Trade-offs

3.1 Ask-Edge Gate (Primary Entry Filter)

```
ask_edge_cents = (p_win - ask_cents/100) × 100  
PASS if ask_edge_cents >= min_ask_edge_cents
```

Tuning	Upside	Downside
Higher min_ask_edge (9→15¢)	Fewer marginal trades; better average edge	Misses fast-moving opportunities; lower fill rate
Lower min_ask_edge (9→5¢)	More entries; better utilization	More cheap/noisy legs; CUT LOSSES dominate
Auto-tune (passive only)	Self-adjusts from win rate	Needs 7+ days, 15+ closed trades; lags regime shifts

Production note: Live underperformed paper partly because resting limits missed ask-edge opportunities — `live_entry.cross_spread` addresses high-edge signals.

3.2 Kelly Sizing

```
b = (1 - ask) / ask  
f_full = (p_win × b - q) / b, clamped [0,1]  
stake = min(bankroll × f_full × kelly_fraction, remaining, bankroll × max_budget_fraction)
```

Variable	Default (BTC hourly)	Trade-off
`kelly_fraction`	0.15 passive / 0.45 aggressive	Higher = larger bets, faster cap consumption, higher variance
`max_budget_fraction_per_entry`	0.25 passive / 0.10 aggressive	Caps single-entry % of hour cap
`max_stake_per_entry_usd`	\$3.50	Hard ceiling; often **not binding** when Kelly sizes small
`max_contracts_per_entry`	6	Prevents huge contract counts on penny legs

Live stake cap:

```
effective_cap = min(max_stake, max_spend × max_budget_fraction, max_spend × 0.35)
```

Dashboard `stake_cap_utilization` reports when cap is binding (≥25% of enters at cap).

3.3 Cross-Spread Live Entry

When `ask_edge >= max(cross_spread_min_edge_cents, min_ask_edge_cents)`:

- Limit at **ask** (taker) → immediate fill intent
- Else passive at **mid** (or bid / bid+1)

Setting	Effect
`cross_spread_min_edge_cents: 12`	Only cross when edge is strong
`aggressive_entries: true`	Cross threshold drops to 10¢
`passive_limit_at: mid`	Weak-edge orders rest below ask (~25% fill rate historically)

Trade-off: Crossing pays the spread (worse entry price) but captures signal; passive saves spread but often misses the hour.

3.4 Tail Entries (1–20¢ default)

Cheap lottery-style brackets. ETH hourly blocks entirely (`tail_entry_block: true`).

Risk	Detail
High variance	Small \$ risk but frequent CUT LOSSES
Performance report	1–20¢ bucket often negative EV live
<code>`tail_entry_min_ask_edge_cents: 15`</code>	Stricter edge required for tail

3.5 Correlation Guard & Barbell

Blocks duplicate tickers, same-side near strikes ($<0.08\%$ gap), opposing threshold hedges.

Barbell exception: YES lower strike + NO higher strike if $\text{gap} \geq \text{barbell_min_strike_gap_pct}$ (20%).

Aggressive mode disables correlation guard → more concurrent correlated risk.

3.6 Scale-In

Add legs to winning positions on same ticker/side.

Requirements: `allow_scale_in`, `unrealized` $\geq \$0.05$, `legs` $< \text{max}$, optional edge improvement.

Passive hourly: up to 4 legs/ticker. **Aggressive:** up to 8.

3.7 Hour-Edge Guards

Guard	Default	Skip reason
Too late	<code>`min_hours_to_settle_for_entry: 0.25`</code>	<code>`too_late_for_new_entries`</code>
Too far	<code>`max_hours_to_settle_for_entry: 1.25`</code>	<code>`too_far_for_new_entries`</code>
Wrong event	ticker validation	<code>`wrong_hour_event:...`</code>

Screenshot example: "skip: too close to hour settle" = within last 15 minutes of hourly contract — **by design** to avoid entries that cannot exit cleanly before settlement.

Part IV — Exit Logic & Trade-offs

4.1 Take Profit (Adaptive / Hybrid)

```
profit_pct = unrealized_usd / cost_usd
effective_tp = base_tp × time_factor × edge_factor × regime_factor
```

Clamped to [min_take_profit_pct, max_take_profit_pct].

Mode	Behavior
`fixed`	Always `take_profit_pct`
`adaptive` / `hybrid`	Tightens TP as hour runs out or edge decays
`trailing` / `hybrid`	Exit on giveback from peak

Hourly defaults: TP 25%, trail arm 8% / \$0.50, min hold 120s.

4.2 Cut Losses vs Cheap Leg Cut

Exit type	Trigger	Typical loss
CHEAP LEG CUT	entry ≤20¢, mark ≤10¢	Small \$ but frequent
CUT LOSSES	Thesis break (hourly alert / 15m monitor)	Larger; live min \$0.20
RECONCILED	Bot/Kalshi mismatch forced close	Variable; was a live pain point

Production finding: CUT LOSSES and RECONCILED exits were largest live loss drivers in early hours.

4.3 Hourly Trial / Slot15 Leg Exits

- **Leg stop:** mark - entry ≤ -leg_stop_loss_cents
- **Leg TP:** mark - entry ≥ +leg_take_profit_cents OR unrealized ≥ leg_take_profit_usd
- **Leg trail:** arm at leg_trail_arm_usd, giveback USD or %

Trial uses tighter stops (4¢) and faster min_hold (15s) — **more churn, faster profit lock.**

4.4 Period Settlement / Rollover

On hour/slot change, open legs force-close at BRTI/ERTI vs strike.

Phantom settlement bug (fixed): Bot logged settlement **before** Kalshi hour ended → inflated P&L. Cleanup voids premature PERIOD SETTLEMENT rows (status=voided).

Part V — Dashboard Guide (Messages, Switches, Screenshot)

5.1 Reading the Bot Panel (Example from Live Hour)

Your screenshot shows a **healthy end-of-hour state**:

Green bar — Settlement index

Settlement index OK: BRTI via CF Benchmarks BRTI (live) · \$58,374.3 — live entries allowed.

Field	Meaning
BRTI	CF Benchmarks Bitcoin Real-Time Index — Kalshi settlement reference
live entries allowed	<code>`live_settlement_index.require_for_live_entries`</code> satisfied
Blocked variant	"Live entry guard: ... feed not active" — live entries paused

Yellow bar — Watching

Watching: BUY NO · \$58,400 or above · live regime (advisory only in FREE mode)
· 0/12 slots · \$0.30 deployable · skip: too close to hour settle

Fragment	API source	Meaning
BUY NO · \$58,400 or above	<code>`entry_watch.signal`</code> + <code>`label`</code>	Best current opportunity from hourly tab
live regime (advisory only in FREE mode)	<code>`entry_watch.regime_allow_trade`</code>	Regime may block in filtered modes; ignored when STRONG/ACTIONABLE both off
0/12 slots	<code>`open_position_count`</code> / <code>`max_concurrent_positions`</code>	No open legs; up to 12 allowed (aggressive hourly)
\$0.30 deployable	<code>`remaining_usd`</code>	Budget left after exposure; low here = nearly fully deployed or post losses
skip: too close to hour settle	<code>`last_skip_reason`</code>	Same as <code>`too_late_for_new_entries`</code> — last 15 min of hour

Last skip line

Last skip: `too_late_for_new_entries`

Machine-readable skip code; dashboard maps to human labels in some views.

Green box — Live reconcile

Live reconcile: OK
Bot legs / contracts: 0 / 0
Kalshi legs / contracts: 0 / 0
Bot at-risk (reconcile): \$0.00
Kalshi exposure (reconcile): \$0.00
Bot open legs match Kalshi for this hour.

Critical safety check. MISMATCH means bot DB and Kalshi exchange disagree — investigate before adding capital.

Mismatch type	Action
`bot_only`	Bot thinks it has legs Kalshi doesn't — may need reconcile-close
`kalshi_only`	Orphan exchange inventory — hygiene may adopt
`orphan_resting_sells`	Cancel stale sell orders on Kalshi

P&L block

BTC · LIVE P&L for hour KXBTCD-26JUN3011 only (resets each new Kalshi hour).
Hour total: +\$1.67 | Realized: +\$1.67 | Unrealized: +\$0.00

Metric	Definition
Hour total	`realized + unrealized` for current `event_ticker` only
Realized	Sum `pnl_usd` on exits with status `filled` or `reconciled`
Unrealized	Mark-to-market on open legs
Resets	New hour = new `event_ticker`; trade log keeps all hours

Note: Hour P&L can disagree with `/api/bots/performance-report` — see Part VII.

5.2 Dashboard Switches & Strategy Impact

Switch	Setting	Strategy effect
Auto-bet	`enabled`	Master on/off
Paper / Live	`mode`	Simulated vs Kalshi orders; live needs credentials
Max at-risk \$	`max_spend_per_hour_usd` / `max_spend_per_slot_usd`	Cap on concurrent open exposure
STRONG	`allow_strong`	Only strong-toned assessments
ACTIONABLE	`allow_actionable`	Moderate + strong actionable
Both OFF → FREE	—	Any actionable signal; regime advisory only (hourly)
Use profits	`use_accumulated_profit`	Wins expand deployable beyond cap
Profit use %	`profit_use_pct`	Fraction of gains redeployed
Auto-refill	`paper_auto_refill`	Paper bankroll resets when exhausted
Aggressive entries	`aggressive_entries`	Kelly 0.45, more entries, no correlation guard, faster cooldowns, cross at 10¢

Aggressive vs Passive summary

	Passive	Aggressive
Kelly	0.15	0.45
Entries/cycle (hourly)	4	5
Concurrent (hourly)	12	12

	Passive	Aggressive
Correlation guard	ON	OFF
Auto-tune	ON	OFF
Reentry CD	120s	30s
Cross-spread threshold	12¢	10¢

When to use aggressive: You accept more legs, faster churn, correlated strikes, and disabled auto-tune — appropriate only after live edge is proven.

5.3 Common Skip Reasons (Complete Reference)

Skip code	User meaning	Config lever
`too_late_for_new_entries`	Last 15m of hour	`min_hours_to_settle_for_entry`
`too_far_for_new_entries`	Wrong future hour book	`max_hours_to_settle_for_entry`
`hour_budget_exhausted`	No deployable \$	Raise cap or wait for exits
`max_concurrent_positions`	All slots full	`max_concurrent_positions` or close legs
`ask_edge_too_low:Nc`	Edge below gate	`min_ask_edge_cents`
`adaptive_bucket_paused:...`	Price bucket memory	`bot_adaptive_calibration`
`cheap_leg_cut_cooldown:...`	Recent cheap-leg stop	`cheap_leg_cut_cooldown_seconds`
`daily_loss_cap`	-\$50 day hit	`bot_risk.daily_loss_cap_usd`
`settlement_index_unavailable`	No BRTI	Index feed / Kalshi auth
`signal_filtered_by_settings`	STRONG/ACTIONABLE filter	Dashboard toggles
`correlated_same_side_strikes`	Too close to open strike	`correlation_guard`
`reentry_cooldown:...`	Recent exit on ticker	`reentry_cooldown_seconds`

5.4 Mode Badge

PAPER / LIVE · FREE — shown when both STRONG and ACTIONABLE are disabled.

Part VI — Paper vs Live: Gaps & Trade-offs

Dimension	Paper	Live
Entry fill	Always at ask	Limit; ~14–58% fill rate observed
Exit fill	At bid	Marketable limit; may rest
Fees	Not modeled fully	Kalshi fees apply
Reconcile	N/A	Required; orphan adoption
Phantom settlement	N/A	Was bug; now voided on cleanup
Bankroll	Auto-refill available	Fixed cap + profit_use

Why paper looked ~\$50 better over same period: Instant fills, no fees, auto-refill, higher effective caps, no reconcile losses.

Capital raise guidance (from production review):

Raise	When	Risk
`max_spend_per_hour` \$15→\$25	Net P&L ≥ 0 over 20+ hours; TP > cuts	Scales losses if edge negative
`max_stake_per_entry` \$3.50→\$5	`stake_cap_utilization` shows binding	Bigger single-leg CUT LOSSES
Don't raise	Cheap-leg cuts dominate; reconcile issues; negative hour P&L	Amplifies known loss modes

Part VII — Stats, P&L & Reporting (Detailed)

7.1 Core P&L Formula

$\text{leg_pnl_usd} = \text{contracts} \times (\text{exit_cents} - \text{entry_cents}) / 100$

Same for YES and NO legs (prices always on owned side).

7.2 Three Stat Layers (Important)

Layer	Used for	Exit statuses counted
hour_summary / slot_summary	Dashboard hour P&L	`filled` + `reconciled`
interval_performance	W/L hour history	`filled` + `reconciled`
performance-report	Tuning, recommendations	**`filled` only**

Known inconsistency: RECONCILED exits count on dashboard but **not** in performance report closed round-trips. When comparing numbers, check exit status.

7.3 hour_summary Computation

SQL aggregate on bot_trades for current event_ticker:

- realized_pnl_usd — sum exit pnl_usd (filled + reconciled)
- total_entered_usd — sum enter cost_usd (can exceed cap with churn)
- open_exposure_usd — sum open position costs
- Scheduler adds unrealized_pnl_usd from live marks
- total_pnl_usd = realized + unrealized

Mode-scoped: If bot is in live mode, paper trades in same hour are excluded from summary.

7.4 Performance Report

GET /api/bots/performance-report:

- Up to 5000 trades per bot (**paper + live mixed**)
- Closed round-trips matched by position_id
- Buckets: entry price, spread, signal, free_mode vs filtered
- Rolling windows: 1h, 2h, 4h, 12h, 24h, 48h
- Recommendations: e.g. "1–20¢ bucket losing — avoid tail strikes"
- stake_cap_utilization for live sizing

7.5 Phantom Settlement Impact

Before fix: rollover logged PERIOD SETTLEMENT at hour boundary while Kalshi positions still open → **fake realized gains/losses**.

After fix: bot_phantom_settlement_cleanup voids rows where created_at < leg_settle_utc.

Triggers: API startup, POST /api/admin/backfill-phantom-settlement.

7.6 entry_settings Snapshot

At each **enter**, bot saves JSON snapshot: mode, free_mode, max_spend, allow_strong/actionable, profit_use_pct.

Used for retrospective "was this FREE mode?" analysis — **not** for live P&L SQL.

7.7 Logging & Backup

Mechanism	What
`bot_trades` SQLite	All enters/exits/resting/failed
`log_backup` every 15m	Full DB snapshots + CSV exports
`audit_trades.jsonl`	Per-trade audit with entry_settings
Paper vs live exports	**Separate** directories (live = tax records)
`scripts/monitor_live_trading.py`	Poll reconcile + stake cap + recent trades

Part VIII — Risks & Known Shortcomings

8.1 Implementation Risks

Risk	Severity	Mitigation
Resting live limits	High	cross_spread for high edge; monitor fill rate
Reconcile drift	High	Live reconcile panel; hygiene loop
Phantom settlement	Medium (fixed)	Startup cleanup; don't trust pre-fix P&L
Wrong-hour entries	Medium (fixed)	max_hours_to_settle, ticker validation
Cheap-leg lottery	Medium	Tail gates; adaptive bucket pause
Daily loss cap	Low	\$50/bot/day auto-stop
Kalshi API circuit	Medium	Pauses entries; doesn't flatten positions
ML drift	Medium	Daily retrain + 6h isotonic refit
Small sample adaptive pause	Low	Probe entries after pause

8.2 Design Limitations

1. **Paper ≠ live simulator** — no resting-order model in paper
2. **Performance report mixes modes** — filter mentally by mode in trade log
3. **Config vs dashboard split** — TP/trail/tail/correlation in yaml only; easy to forget what's active
4. **ETH stricter defaults** — tail blocked; lower caps — don't copy BTC settings blindly
5. **Hourly trial ≠ hourly** — different exit philosophy; separate DB and P&L
6. **6h isotonic refit** — can't fix structural model failure quickly
7. **Aggressive disables guards** — faster trading but more correlated blow-ups

8.3 Operational Checklist

Before raising capital:

- ☐ Live reconcile OK for 24h
 - ☐ Hour P&L positive or flat over 20+ hours
 - ☐ TAKE PROFIT > CUT LOSSES + cheap-leg cuts
 - ☐ stake_cap_utilization shows binding (if raising stake, not cap)
 - ☐ cross_spread entries filling (check trade detail for entry=cross_spread)
 - ☐ No phantom settlement rows after cleanup
 - ☐ Compare dashboard P&L vs Kalshi wallet periodically
-

Part IX — Configuration Encyclopedia

9.1 Global Risk & Tuning

Key	Default	Notes
<code>`bot_risk.daily_loss_cap_usd`</code>	50	Per bot, per calendar day (NY)
<code>`bot_auto_tune.enabled`</code>	true	Passive only
<code>`bot_adaptive_calibration.enabled`</code>	true	Hourly; off for 15m
<code>`live_settlement_index.require_for_live_entries`</code>	true	Blocks live without BRTI/ERTI
<code>`live_resting_exits.enabled`</code>	false	Kalshi bracket orders after cheap-leg fill

9.2 BTC Hourly Bot (``hourly.bot``)

See config.yaml lines 230–310. Key production values:

- `max_spend_per_hour_usd`: 15
- `min_ask_edge_cents`: 9 (entry_strategy)
- `max_stake_per_entry_usd`: 3.50
- `live_entry.cross_spread_min_edge_cents`: 12
- `min_hours_to_settle_for_entry`: 0.25
- `max_hours_to_settle_for_entry`: 1.25

9.3 BTC 15m Bot (``intra_slot.bot``)

- `max_spend_per_slot_usd`: 100 (dashboard often lower)
- `max_stake_per_entry_usd`: 1.50
- `max_entries_per_cycle`: 1 passive
- `paper_max_spread_cents`: 40

9.4 ETH Overrides

- Hourly cap \$10, tail blocked, `min_ask_edge` 10¢, `max_stake` \$1.50
 - 15m: same leg-stop gates as BTC with ETH index ERTI
-

Part X — API & Scheduler Reference

10.1 Bot API Endpoints

Endpoint	Purpose
`GET /api/hourly/bot`	Full status + hour_summary + reconcile
`POST /api/hourly/bot/settings`	Dashboard switches
`GET /api/hourly/bot/trades`	Trade log
`GET /api/hourly/bot/live-reconcile`	Reconcile only
`GET /api/bots/performance-report`	Analytics
`GET /api/bots/risk-status`	Daily caps
`POST /api/admin/backfill-phantom-settlement`	P&L cleanup

(Same patterns for trial, eth, slot15.)

10.2 Scheduler Jobs

Job	Interval
Bot continuous	`poll_seconds` (10)
Candle fetch	1 min
Outcome resolve	1 min
15m prediction	:00/:15/:30/:45 ET
Hourly lock	:05 ET
Hourly late call	:45 ET
2nd Chance	:04/:19/:34/:49 ET
Model retrain	2:00 AM ET daily
Auto-tune	3:00 AM ET daily
Adaptive calibration	30 min
Log backup	15 min

Part XI — Module Reference

Module	Role
`hourly_bot.py` / `slot15_bot.py`	Main trading loops
`entry_strategy.py`	Kelly, edge, ranking, correlation
`bot_profit_exit.py`	TP, trail, cheap leg, leg stops
`bot_budget.py`	Deploy bankroll math
`live_entry_price.py`	Cross-spread pricing
`live_position_sync.py`	Reconcile, adopt, exit
`bot_adaptive_calibration.py`	Bucket memory
`bot_performance_report.py`	Analytics
`stake_cap_utilization.py`	max_stake binding report
`bot_phantom_settlement_cleanup.py`	Void bad settlement rows
`scheduler/loop.py`	All jobs + status enrichment

Appendix A — Glossary

Term	Definition
Ask-edge	Model win probability minus ask-implied probability (¢)
BRTI / ERTI	CF Benchmarks real-time indices for BTC/ETH settlement
FREE mode	STRONG and ACTIONABLE both off — widest entry filter
Cross-spread	Live limit at ask when edge is high
Cheap leg	$\text{Entry} \leq \text{cheap_leg_max_entry_cents}$ (default 20¢)
Reconcile	Aligning bot DB with Kalshi exchange state
Phantom settlement	Premature PERIOD SETTLEMENT before real settle
event_ticker	Kalshi hour/slot identifier (e.g. KXBTCD-26JUN3011)
Probe	Adaptive bucket trial entries after pause

Appendix B — Entry Decision Flow

enabled? → continuous? → daily cap / circuit OK?
→ settlement index (live)?
→ hour-edge guards (too late / too far / wrong event)?
→ regime (filtered modes only)?
→ rank candidates (EV composite hourly)
→ per candidate:
 cooldowns → adaptive bucket → ask-edge → tail gate
 → correlation → budget → Kelly stake → live cap
 → paper fill OR live order (cross or passive)

Appendix C — Exit Decision Flow

```
for each open leg:
    → cheap leg cut? (mark at floor)
    → adaptive TP / trail?
    → thesis CUT LOSSES / TAKE PROFIT? (hourly standard)
    → leg stop/TP/trail? (trial / 15m)
    → paper fill OR live exit sell
on rollover:
    → force settlement exit at BRTI/ERTI vs strike
```

End of comprehensive report. Regenerate PDF: `python3 scripts/generate_design_report_pdf.py`